

✓ Pedestrian Detection using Fast R-CNN

Problem Statement

Detect pedestrians in real-world images using a region-based object detection model and visualize predictions.

Objectives

Understand Fast R-CNN pipeline

Apply transfer learning

Perform object detection on real dataset

Visualize predictions and analyze results

Dataset

Penn-Fudan Pedestrian Dataset

Description:

Real street images

Annotated pedestrians using masks

✓ STEP 1: Install Libraries

```
!pip install torch torchvision opencv-python matplotlib
```

```
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.10.0+cpu)
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (0.25.0+cpu)
Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (4.13.0.92)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.29.0)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from torchvision) (2.0.2)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from torchvision) (11.3.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied:ycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.1)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.37.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)
```

Installs required libraries:

PyTorch → model

Torchvision → Faster R-CNN

OpenCV → visualization

Matplotlib → display

✓ STEP 2: Download Dataset

```
!wget https://www.cis.upenn.edu/~jshi/ped_html/PennFudanPed.zip
!unzip PennFudanPed.zip
```

```
inflation: PennFudanPed/PNGImages/PennFudanPed00046.png
```

```
inflating: PennFudanPed/PNGImages/PennPed00040.png
inflating: PennFudanPed/PNGImages/PennPed00047.png
inflating: PennFudanPed/PNGImages/PennPed00048.png
inflating: PennFudanPed/PNGImages/PennPed00049.png
inflating: PennFudanPed/PNGImages/PennPed00050.png
inflating: PennFudanPed/PNGImages/PennPed00051.png
inflating: PennFudanPed/PNGImages/PennPed00052.png
inflating: PennFudanPed/PNGImages/PennPed00053.png
inflating: PennFudanPed/PNGImages/PennPed00054.png
inflating: PennFudanPed/PNGImages/PennPed00055.png
inflating: PennFudanPed/PNGImages/PennPed00056.png
inflating: PennFudanPed/PNGImages/PennPed00057.png
inflating: PennFudanPed/PNGImages/PennPed00058.png
inflating: PennFudanPed/PNGImages/PennPed00059.png
inflating: PennFudanPed/PNGImages/PennPed00060.png
inflating: PennFudanPed/PNGImages/PennPed00061.png
inflating: PennFudanPed/PNGImages/PennPed00062.png
inflating: PennFudanPed/PNGImages/PennPed00063.png
inflating: PennFudanPed/PNGImages/PennPed00064.png
inflating: PennFudanPed/PNGImages/PennPed00065.png
inflating: PennFudanPed/PNGImages/PennPed00066.png
inflating: PennFudanPed/PNGImages/PennPed00067.png
inflating: PennFudanPed/PNGImages/PennPed00068.png
inflating: PennFudanPed/PNGImages/PennPed00069.png
inflating: PennFudanPed/PNGImages/PennPed00070.png
inflating: PennFudanPed/PNGImages/PennPed00071.png
inflating: PennFudanPed/PNGImages/PennPed00072.png
inflating: PennFudanPed/PNGImages/PennPed00073.png
inflating: PennFudanPed/PNGImages/PennPed00074.png
inflating: PennFudanPed/PNGImages/PennPed00075.png
inflating: PennFudanPed/PNGImages/PennPed00076.png
inflating: PennFudanPed/PNGImages/PennPed00077.png
inflating: PennFudanPed/PNGImages/PennPed00078.png
inflating: PennFudanPed/PNGImages/PennPed00079.png
inflating: PennFudanPed/PNGImages/PennPed00080.png
inflating: PennFudanPed/PNGImages/PennPed00081.png
inflating: PennFudanPed/PNGImages/PennPed00082.png
inflating: PennFudanPed/PNGImages/PennPed00083.png
inflating: PennFudanPed/PNGImages/PennPed00084.png
inflating: PennFudanPed/PNGImages/PennPed00085.png
inflating: PennFudanPed/PNGImages/PennPed00086.png
inflating: PennFudanPed/PNGImages/PennPed00087.png
inflating: PennFudanPed/PNGImages/PennPed00088.png
inflating: PennFudanPed/PNGImages/PennPed00089.png
inflating: PennFudanPed/PNGImages/PennPed00090.png
inflating: PennFudanPed/PNGImages/PennPed00091.png
inflating: PennFudanPed/PNGImages/PennPed00092.png
inflating: PennFudanPed/PNGImages/PennPed00093.png
inflating: PennFudanPed/PNGImages/PennPed00094.png
inflating: PennFudanPed/PNGImages/PennPed00095.png
inflating: PennFudanPed/PNGImages/PennPed00096.png
inflating: PennFudanPed/readme.txt
```

Explanation:

Downloads and extracts dataset with:

Images

Masks (used to create bounding boxes)

✓ STEP 3: Load and Prepare Dataset

```
import numpy as np
from PIL import Image
import torch

def get_bbox_from_mask(mask):
    objs = np.unique(mask)
    objs = objs[1:]

    boxes = []
    for obj in objs:
        pos = np.where(mask == obj)
        xmin, xmax = np.min(pos[1]), np.max(pos[1])
        ymin, ymax = np.min(pos[0]), np.max(pos[0])
        boxes.append([xmin, ymin, xmax, ymax])

    return boxes

img = Image.open("PennFudanPed/PNGImages/FudanPed00001.png")
mask = np.array(Image.open("PennFudanPed/PedMasks/FudanPed00001_mask.png"))

boxes = get_bbox_from_mask(mask)
```

```
img_tensor = torch.tensor(np.array(img)).permute(2,0,1)/255.0
```

Explanation:

Converts mask → bounding boxes

Converts image → tensor

Prepares data for model

The dataset gives:

Images

Masks (pixel-level annotations)

But Fast R-CNN needs:

Bounding boxes (rectangles around objects)

So we convert:

Mask → Bounding Boxes → Model Input

Reads the image

→ actual picture (pedestrian scene)

Reads the mask

→ shows where each person is (highlighted regions)

Finds individual objects

→ each person has a unique value in the mask

Creates bounding boxes

→ draws a rectangle around each person

Converts image into tensor

→ model understands only numerical data

Mask = pixel-level information

Bounding box = object-level information

TRANSFER LEARNING CONCEPT

We use a pretrained model instead of training from scratch.

Why?

Already trained on large dataset (COCO)

Knows general features (edges, shapes)

We reuse this knowledge

This is called Transfer Learning

✓ STEP 4: Load Fast R-CNN Model

```
import torchvision
from torchvision.models.detection import fasterrcnn_resnet50_fpn

model = fasterrcnn_resnet50_fpn(pretrained=True)
model.eval()
```

```

    )
    )
    (fpn): FeaturePyramidNetwork(
      (inner_blocks): ModuleList(
        (0): Conv2dNormActivation(
          (0): Conv2d(256, 256, kernel_size=(1, 1), stride=(1, 1))
        )
        (1): Conv2dNormActivation(
          (0): Conv2d(512, 256, kernel_size=(1, 1), stride=(1, 1))
        )
        (2): Conv2dNormActivation(
          (0): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1))
        )
        (3): Conv2dNormActivation(
          (0): Conv2d(2048, 256, kernel_size=(1, 1), stride=(1, 1))
        )
      )
      (layer_blocks): ModuleList(
        (0-3): 4 x Conv2dNormActivation(
          (0): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        )
      )
      (extra_blocks): LastLevelMaxPool()
    )
  )
  (rpn): RegionProposalNetwork(
    (anchor_generator): AnchorGenerator()
    (head): RPNHead(
      (conv): Sequential(
        (0): Conv2dNormActivation(
          (0): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
          (1): ReLU(inplace=True)
        )
      )
      (cls_logits): Conv2d(256, 3, kernel_size=(1, 1), stride=(1, 1))
      (bbox_pred): Conv2d(256, 12, kernel_size=(1, 1), stride=(1, 1))
    )
  )
  (roi_heads): RoIHeads(
    (box_roi_pool): MultiScaleRoIAlign(featmap_names=['0', '1', '2', '3'], output_size=(7, 7), sampling_ratio=2)
    (box_head): TwoMLPHead(
      (fc6): Linear(in_features=12544, out_features=1024, bias=True)
      (fc7): Linear(in_features=1024, out_features=1024, bias=True)
    )
    (box_predictor): FastRCNNPredictor(

```

Explanation:

Loads pretrained Fast R-CNN

Uses ResNet50 backbone

.eval() → inference mode

✓ STEP 5: Perform Inference

```

from torchvision.transforms import functional as F

img_tensor = F.to_tensor(img)

with torch.no_grad():
    prediction = model([img_tensor])

```

Explanation:

Converts image → tensor

Passes image to model

Gets predictions

torch.no_grad() → faster inference

✓ STEP 6: Understand Output

```

print(prediction)

[{'boxes': tensor([[419.6865, 170.0683, 536.0842, 493.7453],
                  [159.0727, 180.3605, 298.8194, 434.4604],
                  [439.7836, 222.6208, 452.0138, 271.8359],

```

```
[444.3562, 224.4628, 456.1511, 265.5336],  
[437.7808, 226.5965, 446.2904, 271.2691]]], 'labels': tensor([ 1,  1, 32, 32, 32]), 'scores': tensor([0.9997, 0.9996])
```

Explanation:**Output contains:**

boxes → object locations

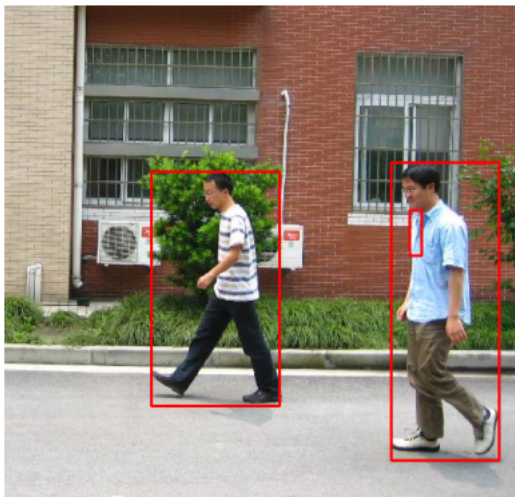
labels → class IDs

scores → confidence

✓ STEP 7: Visualization

```
import cv2  
import matplotlib.pyplot as plt  
  
img_np = np.array(img)  
  
for i, box in enumerate(prediction[0]['boxes']):  
    score = prediction[0]['scores'][i]  
  
    if score > 0.5:  
        x1, y1, x2, y2 = map(int, box.tolist())  
        cv2.rectangle(img_np, (x1,y1), (x2,y2), (255,0,0), 2)  
  
plt.imshow(img_np)  
plt.title("Fast R-CNN Detection")  
plt.axis('off')  
  
(np.float64(-0.5), np.float64(558.5), np.float64(535.5), np.float64(-0.5))
```

Fast R-CNN Detection

**Explanation:**

Draws bounding boxes

Filters low-confidence detections

Displays final result

✓ STEP 8: Results Visualization (Confidence Scores)

```
import cv2  
import numpy as np  
import matplotlib.pyplot as plt  
  
img_np = np.array(img)  
  
for i, box in enumerate(prediction[0]['boxes']):  
    score = prediction[0]['scores'][i]  
  
    if score > 0.5:  
        x1, y1, x2, y2 = map(int, box.tolist())
```

```

# Draw bounding box
cv2.rectangle(img_np, (x1, y1), (x2, y2), (255, 0, 0), 2)

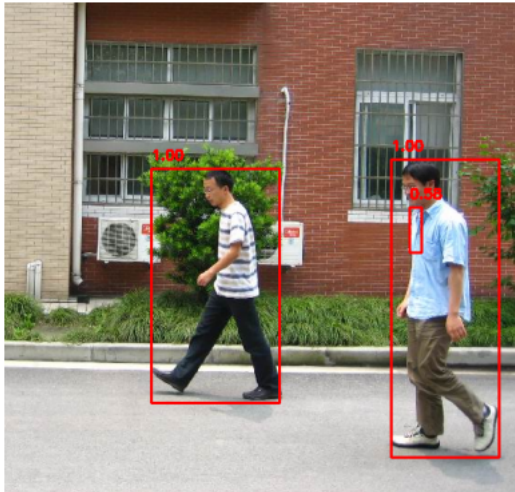
# Put confidence score text
label = f"{score:.2f}"
cv2.putText(
    img_np, label,
    (x1, y1 - 10),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.5,
    (255, 0, 0),
    2
)

plt.imshow(img_np)
plt.title("Fast R-CNN Detection with Confidence Scores")
plt.axis('off')

(np.float64(-0.5), np.float64(558.5), np.float64(535.5), np.float64(-0.5))

```

Fast R-CNN Detection with Confidence Scores



Explanation:

Shows confidence of detections
Helps analyze model performance

✓ STEP 9: Measure Inference Time

```

import time

start = time.time()

with torch.no_grad():
    model([img_tensor])

frcnn_time = time.time() - start # ✅ STORE IT

print("Fast R-CNN Time:", frcnn_time)

Fast R-CNN Time: 8.002482891082764

```

✓ STEP 10: Load YOLO Model

```
!pip install ultralytics torch torchvision opencv-python matplotlib
```

```

Collecting ultralytics
  Downloading ultralytics-8.4.46-py3-none-any.whl.metadata (39 kB)
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.10.0+cpu)
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (0.25.0+cpu)
Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (4.13.0.92)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: numpy>=1.23.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.0.2)
Requirement already satisfied: pillow>=7.1.2 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (11.3.0)
Requirement already satisfied: pyyaml>=5.3.1 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (6.0.3)
Requirement already satisfied: requests>=2.23.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.32.4)
Requirement already satisfied: scipy>=1.4.1 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (1.16.3)

```

```
Requirement already satisfied: psutil>=5.8.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (5.9.5)
Requirement already satisfied: polars>=0.20.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (1.35.2)
Collecting ultralytics-thop>=2.0.18 (from ultralytics)
  Downloading ultralytics_thop-2.0.19-py3-none-any.whl.metadata (14 kB)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.29.0)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.1)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: polars-runtime-32==1.35.2 in /usr/local/lib/python3.12/dist-packages (from polars>=0.20.0->ultralytics) (1.35.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics) (3.10.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics) (2025.1.1)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch) (3.0.3)
Downloading ultralytics-8.4.46-py3-none-any.whl (1.2 MB)
  1.2/1.2 MB 20.5 MB/s eta 0:00:00
Downloading ultralytics_thop-2.0.19-py3-none-any.whl (28 kB)
Installing collected packages: ultralytics-thop, ultralytics
Successfully installed ultralytics-8.4.46 ultralytics-thop-2.0.19
```

```
from ultralytics import YOLO
```

```
yolo_model = YOLO("yolov8n.pt")
```

```
Creating new Ultralytics Settings v0.0.6 file 
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see https://docs.ultralytics.com
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8n.pt to 'yolov8n.pt': 100%  6.1
```

Explanation

Loads a pretrained YOLO model

Already trained on large dataset (general object knowledge)

We will use it for inference (prediction)

This also uses transfer learning

Model already knows features

We apply it directly to our dataset

STEP 11: Perform Inference on Same Image

```
results = yolo_model.predict("PennFudanPed/PNGImages/FudanPed00001.png")
```

```
image 1/1 /content/PennFudanPed/PNGImages/FudanPed00001.png: 640x640 2 persons, 242.9ms
Speed: 13.8ms preprocess, 242.9ms inference, 9.6ms postprocess per image at shape (1, 3, 640, 640)
```

Explanation

Passes image to YOLO

Model detects objects in one pass

Stores results internally

STEP 12: Visualize YOLO with Confidence Scores

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("PennFudanPed/PNGImages/FudanPed00001.png")
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```

for result in results:
    boxes = result.bboxes.xyxy
    scores = result.bboxes.conf

    for box, score in zip(boxes, scores):
        x1, y1, x2, y2 = map(int, box.tolist())

        # Draw box
        cv2.rectangle(img, (x1,y1), (x2,y2), (0,255,0), 2)

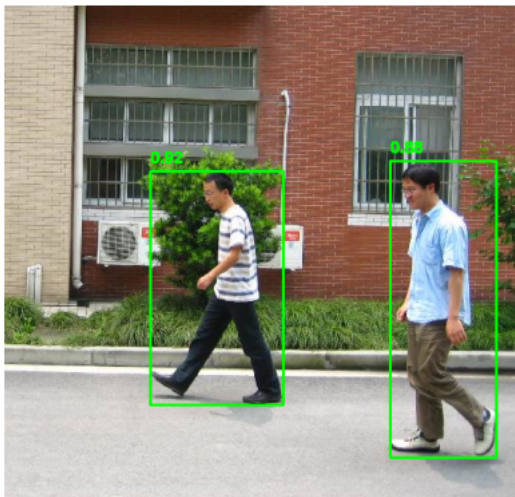
        # Add confidence
        label = f"{score:.2f}"
        cv2.putText(img, label, (x1, y1-10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0,255,0), 2)

plt.imshow(img)
plt.title("YOLO Detection with Confidence")
plt.axis('off')

```

```
(np.float64(-0.5), np.float64(558.5), np.float64(535.5), np.float64(-0.5))
```

YOLO Detection with Confidence



Explanation

Draws bounding boxes (green)

Adds confidence score

Similar visualization as Fast R-CNN

✓ STEP 13: Measure YOLO Inference Time

```

import time

start = time.time()

yolo_model.predict("PennFudanPed/PNGImages/FudanPed00001.png", verbose=False)

yolo_time = time.time() - start
print("YOLO Time:", yolo_time)

```

```
YOLO Time: 0.22640252113342285
```

Explanation

Measures how fast YOLO predicts

Used for comparison

✓ STEP 15: Comparison Graph

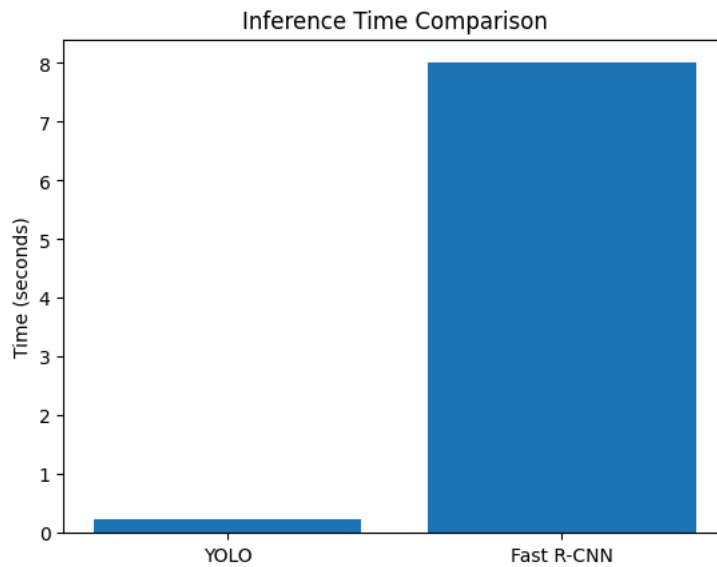
```

import matplotlib.pyplot as plt

models = ["YOLO", "Fast R-CNN"]
times = [yolo_time, frcnn_time]

```

```
plt.bar(models, times)
plt.title("Inference Time Comparison")
plt.ylabel("Time (seconds)")
plt.show()
```



Explanation

Visual comparison of speed

YOLO bar → smaller (faster)

Fast R-CNN → larger (slower)

****KEY OBSERVATION ****

YOLO:

Detects objects in one step

Faster inference

Fast R-CNN:

Uses region proposals + classification

More computation → slower

YOLO → best for real-time applications

Fast R-CNN → best for high-accuracy tasks

“YOLO performs object detection in a single pass making it faster, while Fast R-CNN uses a multi-stage pipeline resulting in higher accuracy but slower inference.”